

Codeigniter4 - Webservice Server

Pada tutorial sebelumnya, pada project telah berhasil menggunakan data ongkir yang disediakan melalui webservice Raja Ongkir (project Toko sebagai webservice client). Data ongkir tersebut digunakan pada saat proses checkout transaksi pembelian user, yang kemudian semua data transaksi beserta detailnya disimpan pada database.

Siapkan project, lakukan clone project (git clone) **jika membutuhkan**. Langkah-langkah untuk clone silakan lihat tutorial sebelumnya. ▶

Sebelum coding atau melakukan perubahan pada project, dibiasakan melakukan **git pull** dulu pada terminal project di VS Code supaya mendapatkan isi project yang paling up-to-date, terutama bagi yang melakukan coding di lebih dari satu perangkat (misalnya di laptop pribadi dan di komputer lab kampus). Perintahnya

```
git pull origin main
```



▶ Menambahkan menu History

- Supaya user bisa melihat riwayat pembelian yang sudah pernah dilakukan, buat satu menu baru untuk menampilkan data tersebut. Misalnya nama menunya adalah History. Tambahkan route nya pada file app/Config/Routes.php.

```
$routes->get('history', 'TransaksiController::history', ['filter' => 'auth']);
```

```

14 $routes->group('produk', ['filter' => 'auth'], function ($routes) {
15     $routes->post('/', 'ProdukController::create');
16     $routes->post('edit/{any}', 'ProdukController::edit/$1');
17     $routes->get('delete/{any}', 'ProdukController::delete/$1');
18     $routes->get('download', 'ProdukController::download');
19 });
20
21
22 $routes->group('keranjang', ['filter' => 'auth'], function ($routes) {
23     $routes->get('/', 'TransaksiController::index');
24     $routes->post('/', 'TransaksiController::cart_add');
25     $routes->post('edit', 'TransaksiController::cart_edit');
26     $routes->get('delete/{any}', 'TransaksiController::cart_delete/$1');
27     $routes->get('clear', 'TransaksiController::cart_clear');
28     $routes->get('checkout', 'TransaksiController::checkout');
29 });
30
31 $routes->get('checkout', 'TransaksiController::checkout', ['filter' => 'auth']);
32 $routes->post('buy', 'TransaksiController::buy', ['filter' => 'auth']);
33 $routes->get('history', 'TransaksiController::history', ['filter' => 'auth']);
34
35 $routes->get('ajax/destinations', 'TransaksiController::destinations', ['filter' => 'auth']);
36 $routes->get('ajax/costs', 'TransaksiController::costs', ['filter' => 'auth']);

```

- Menu History ini dapat diakses melalui sidebar menu. Buka file app/Views/components/sidebar.php, lalu tambahkan menu History di bawah menu Produk

```

<li class="nav-item">
    <a class="nav-link <?php echo (uri_string() == 'history') ? "" : "collapsed" ?>" href="hist
        <i class="bi bi-person"></i>
        <span>History</span>
    </a>
</li><!-- End History Nav -->

```

```

2 <aside id="sidebar" class="sidebar">
3     <ul class="sidebar-nav" id="sidebar-nav">
4         <?php
5             if (session()->get('role') == 'admin') {
6                 <?php
7                     <li class="nav-item">
8                         <a class="nav-link <?php echo (uri_string() == 'produk') ? "" : "collapsed" ?>" href="#">
9                             <i class="bi bi-receipt"></i>
10                            <span>Produk</span>
11                        </a>
12                    </li><!-- End Produk Nav -->
13                <?php
14                }
15                <?php
16
17                <li class="nav-item">
18                    <a class="nav-link <?php echo (uri_string() == 'history') ? "" : "collapsed" ?>" href="#">
19                        <i class="bi bi-person"></i>
20                        <span>History</span>
21                    </a>
22                </li><!-- End History Nav -->
23
24                <li class="nav-item">
25                    <a class="nav-link <?php echo (uri_string() == 'profile') ? "" : "collapsed" ?>" href="#">
26                        <i class="bi bi-person"></i>
27                    </a>
28                </li>
29            }
30        <?php
31    }
32
33    </ul>
34 </aside>

```

- Sesuai dengan route yang dibuat pada langkah no 1 tadi, menu History menggunakan controller TransaksiController. Buka file app/Controllers/TransaksiController.php, sesuai dengan route yang dibuat pada langkah no 1 tadi maka tambahkan function baru bernama history(), letakkan saja di bawah fungsi buy().

```

public function history()
{
    $username = session()->get('username');
}

```

```

$transactions = $this->transactionModel->where('username', $username)->findAll();
$transactionIds = array_column($transactions, 'id');

$products = $this->transactionDetailModel->getProductsByTransactionIds($transactionIds);

$data = [
    'username'      => $username,
    'transactions' => $transactions,
    'products'      => $products
];

return view('v_history', $data);
}

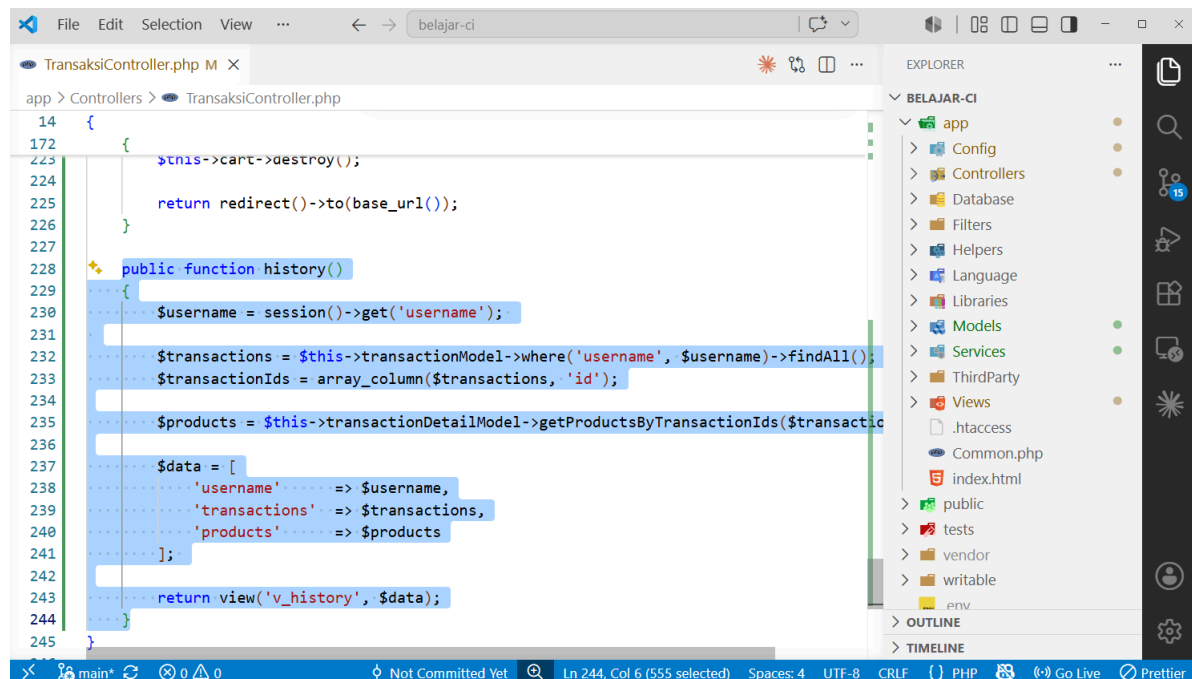
```

Pada fungsi ini, pertama ambil variable username dari session yang tersimpan lalu ambil semua data transaksi yang pernah dilakukan user melalui fungsi where() dan findAll() pada transactionModel. Tampung semua id transaksinya pada sebuah variable array.

Kemudian ambil semua data detail transaksi berdasarkan semua id transaksi tersebut melalui fungsi getProductsByTransactionIds() pada transactionDetailModel (saat ini fungsi tersebut belum ada).

Kenapa harus memanggil fungsi dari transactionDetailModel? Karena logic query database yang akan dibuat itu cukup kompleks, maka sebaiknya diletakkan pada sebuah fungsi di dalam modelnya. Sehingga pada controller hanya fokus menerima request, memanggil model, dan mengirim hasil datanya ke view.

Lalu terakhir, kirimkan data username, transaksi, dan detail transaksi ke view v_history (saat ini file tersebut belum ada).



Kalau project masih kecil, logic boleh saja tetap di controller. Tetapi jika project akan berkembang, memindahkan query dan pengolahan data ke model membuat controller lebih bersih dan lebih mudah di-maintain.

- Tambahkan fungsi transactionDetailModel() pada file app\Models\TransactionDetailModel.php.

```

public function getProductsByTransactionIds(array $transactionIds)
{
    if (empty($transactionIds)) {
        return [];
    }

    $details = $this->select('transaction_detail.*, product.nama, product.harga, product.foto')

```

```

->join('product', 'transaction_detail.product_id = product.id')
->whereIn('transaction_id', $transactionIds)
->findAll();

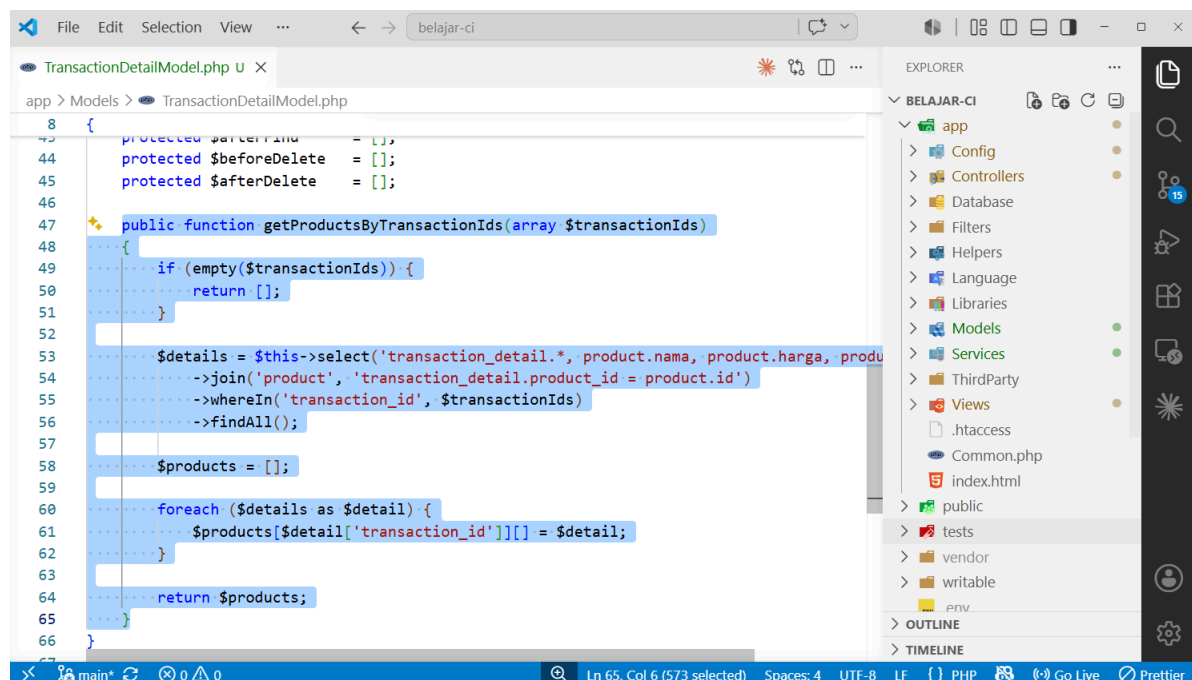
$products = [];

foreach ($details as $detail) {
    $products[$detail['transaction_id']] = $detail;
}

return $products;
}

```

Pada fungsi ini, lakukan pengambilan data detail transaksi berdasarkan semua id transaksi yang pernah dilakukan user. Kemudian data detail transaksi tersebut dikelompokkan dalam sebuah array dengan menggunakan id transaksi sebagai index arraynya.



- Data transaksi dan detailnya yang sudah didapatkan pada function history() di controller TransaksiController, siap untuk ditampilkan pada sebuah view.

Siapkan sebuah file view baru bernama app/Views/v_history.php. Tampilkan data transaksi pada sebuah tabel. Untuk data detail produk yang dibeli untuk setiap transaksinya, tampilkan dalam elemen modal yang akan muncul jika tombol Detail diklik.

```

<?= $this->extend('layout') ?>
<?= $this->section('content') ?>
History Transaksi Pembelian <strong><?= $username ?></strong>
<hr>
<div class="table-responsive">
    <!-- Table with striped rows -->
    <table class="table datatable">
        <thead>
            <tr>
                <th scope="col">#</th>
                <th scope="col">ID Pembelian</th>
                <th scope="col">Waktu Pembelian</th>
                <th scope="col">Total Bayar</th>
                <th scope="col">Alamat</th>
                <th scope="col">Status</th>
                <th scope="col"></th>
            </tr>
        </thead>
    </table>

```

```

<tbody>
    <?php
    if (!empty($transactions)) :
        foreach ($transactions as $index => $item) :
            ?>
                <tr>
                    <th scope="row"><? $index + 1 ?></th>
                    <td><? $item['id'] ?></td>
                    <td><? $item['created_at'] ?></td>
                    <td><? number_to_currency($item['total_harga'], 'IDR') ?></td>
                    <td><? $item['alamat'] ?></td>
                    <td>
                        <? ($item['status'] == "1")
                        ? '<span class="badge bg-success">Sudah Selesai</span>'
                        : '<span class="badge bg-warning">Belum Selesai</span>' ?>
                    </td>
                    <td>
                        <button type="button" class="btn btn-success btn-sm" data-bs-toggle="toggle" data-target="#detailModal-?>
                        Detail
                        </button>
                    </td>
                </tr>
            <?php
            endforeach;
        endif;
    ?>
</tbody>
</table>
<!-- End Table with stripped rows -->
</div>

<?php if (!empty($transactions)) : ?>
    <?php foreach ($transactions as $item) : ?>
        <!-- Detail Modal Begin -->
        <div class="modal fade" id="detailModal-<? $item['id'] ?>" tabindex="-1">
            <div class="modal-dialog modal-dialog-centered">
                <div class="modal-content">
                    <div class="modal-header">
                        <h5 class="modal-title">Detail Transaksi #<? $item['id'] ?></h5>
                        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
                    </div>
                    <div class="modal-body">
                        <?php if (!empty($products[$item['id']])) : ?>
                            <?php foreach ($products[$item['id']] as $index2 => $item2) : ?>
                                <? $index2 + 1 . ">" ?>

                                <?php
                                $imagePath = FCPATH . 'img/' . $item2['foto'];

                                if (!empty($item2['foto']) && file_exists($imagePath)) :
                                    ?>
                                        <div class="my-2">
                                            
                                        </div>
                                    <?php endif; ?>

                                <strong><? $item2['nama'] ?></strong>
                                <? number_to_currency($item2['harga'], 'IDR') ?>
                                <br>
                                <? "(" . $item2['jumlah'] . " pcs)" ?><br>
                                <? number_to_currency($item2['subtotal_harga'], 'IDR') ?>
                                <hr>
                            <?php endforeach; ?>
                        <?php endif; ?>
                        Ongkir <? number_to_currency($item['ongkir'], 'IDR') ?>
                    </div>
                </div>
            </div>
        </div>
    <?php endforeach; ?>
</if>
</div>

```

```

        </div>
    </div>
</div>
<!-- Detail Modal End -->
<?php endforeach; ?>
<?php endif; ?>
<?= $this->endSection() ?>

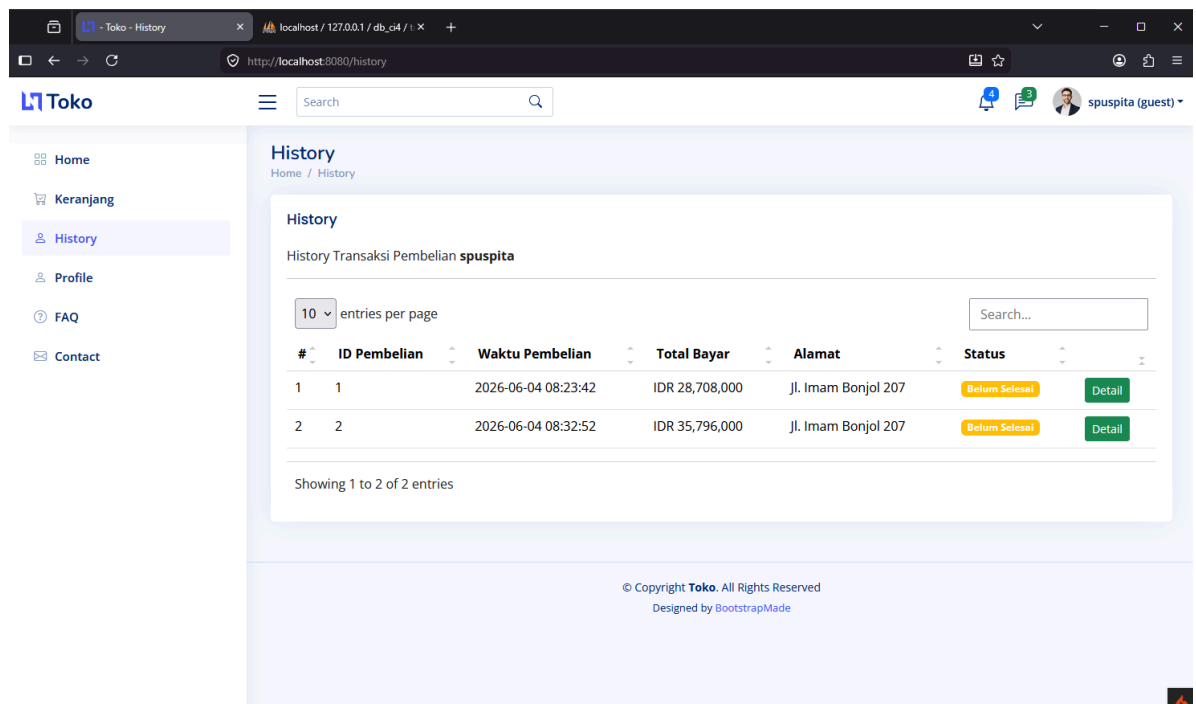
```

```

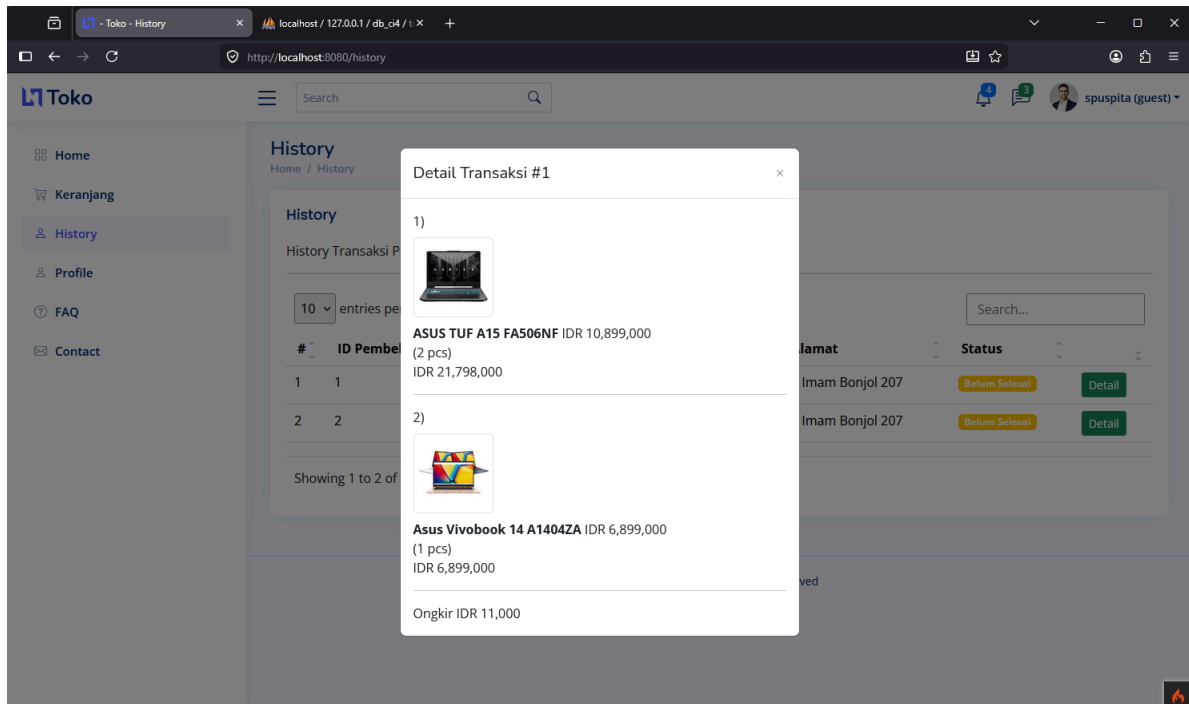
1 <?= $this->extend('layout') ?>
2 <?= $this->section('content') ?>
3 History Transaksi Pembelian <strong><?= $username ?></strong>
4 <hr>
5 <div class="table-responsive">
6     <!-- Table with striped rows -->
7     <table class="table datatable">
8         <thead>
9             <tr>
10                <th scope="col">#</th>
11                <th scope="col">ID Pembelian</th>
12                <th scope="col">Waktu Pembelian</th>
13                <th scope="col">Total Bayar</th>
14                <th scope="col">Alamat</th>
15                <th scope="col">Status</th>
16            </tr>
17        </thead>
18        <tbody>
19            <?php
20            if (!empty($transactions)) :
21                <?php foreach ($transactions as $index => $item) :
22                    <tr>
23                        <th scope="row"><?= $index + 1 ?></th>

```

Hasil tampilan menu History.



Jika tombol Detail diklik



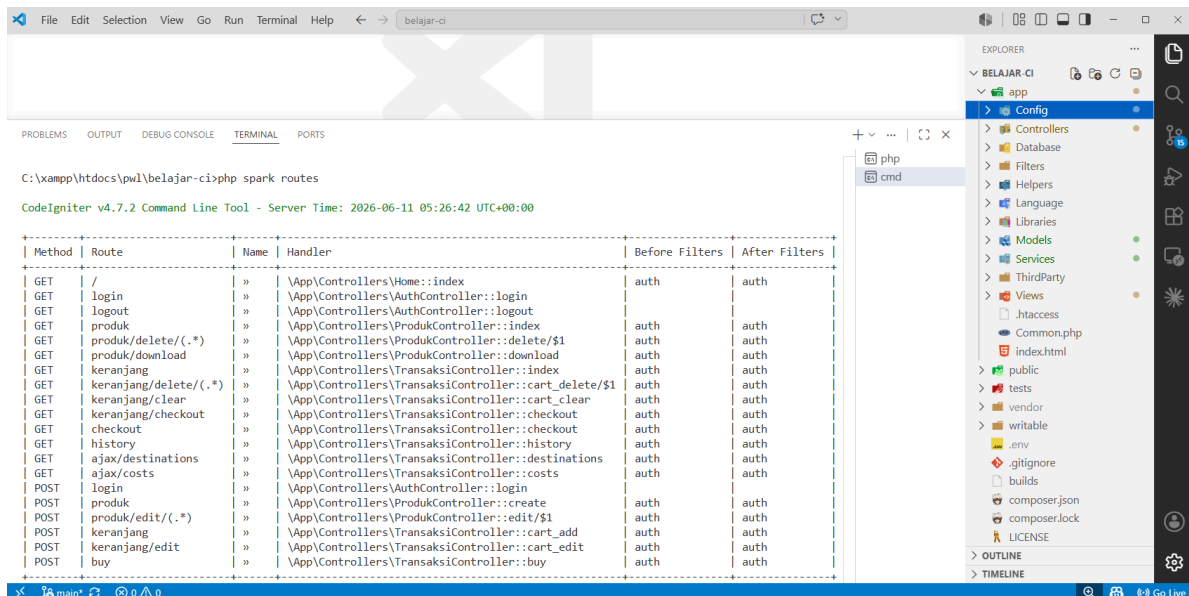
► Membuat endpoint data produk dengan RESTful Resource Handling

- Alur sistem project Toko sudah cukup lengkap hingga saat ini. Mulai dari otentikasi user login, manajemen data produk, keranjang belanja, hingga halaman checkout.

Coba lihat semua routes yang sudah dimiliki project Toko saat ini, dengan menjalankan perintah ini pada terminal baru

```
php spark route
```

Satu baris code rute yang ada di file app\Config\Routes.php menghasilkan sebuah rute pada project Toko.



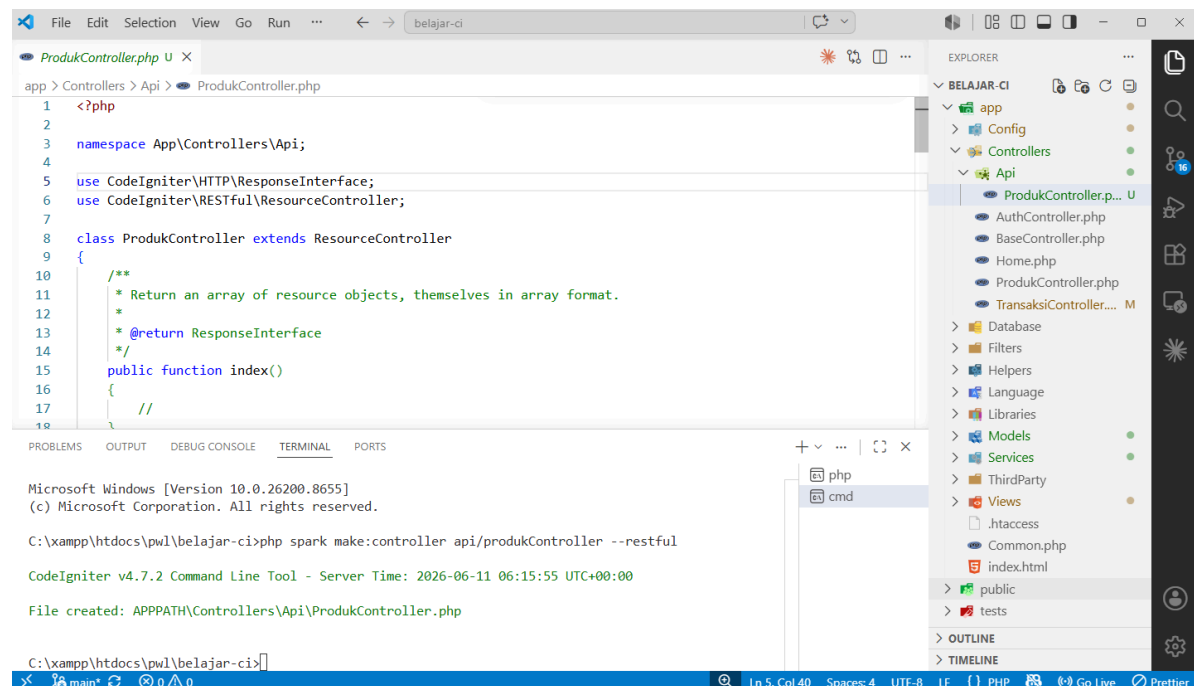
- Selanjutnya, buat project Toko ini supaya bisa menyediakan web service agar data produk, stok, pelanggan, dan transaksi dapat digunakan oleh aplikasi lain, seperti mobile app, website, kasir, gudang, dan dashboard, tanpa harus memberikan akses langsung ke database Toko. Jadi perannya project Toko adalah sebagai Webservice Server seperti RajaOngkir yang menyediakan data daerah tujuan dan layanan ongkos kirim pada tutorial sebelumnya.

Webservice yang disediakan misalnya ada 2, yang pertama untuk manajemen data produk dan yang kedua adalah untuk mendapatkan data transaksi pada range tanggal tertentu.

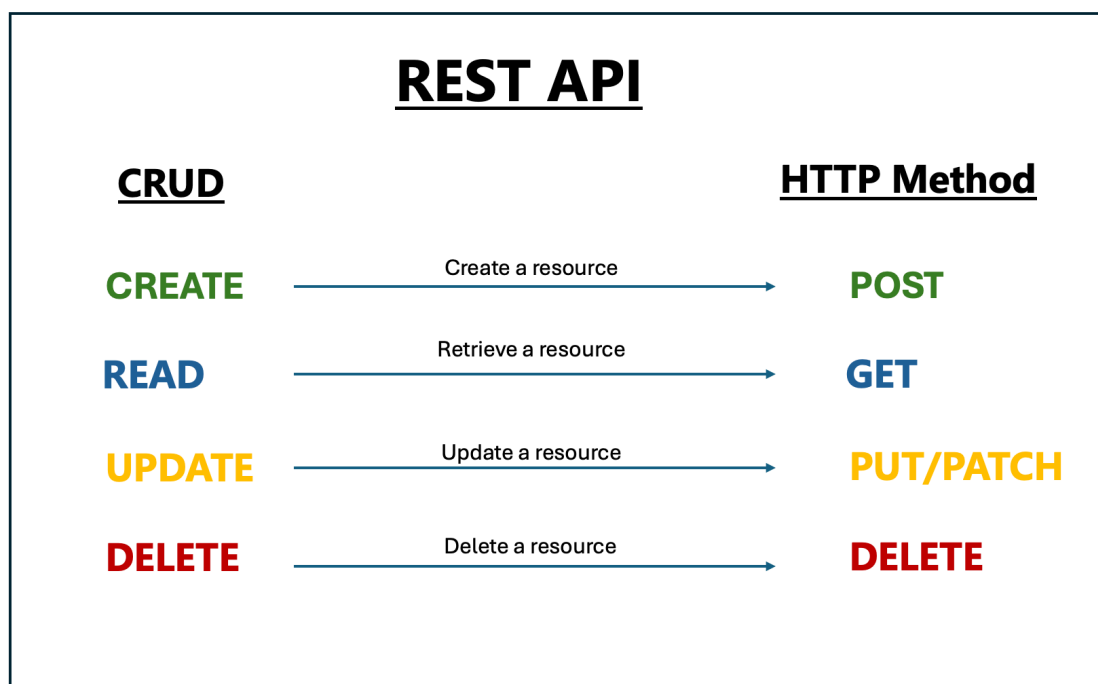
Untuk webservice data produk, karena akan digunakan untuk manajemen data (CRUD) maka buat rutenya menggunakan jenis controller dengan tipe restful.

https://codeigniter.com/user_guide/incoming/restful.html

```
php spark make:controller api/produkController --restful
```

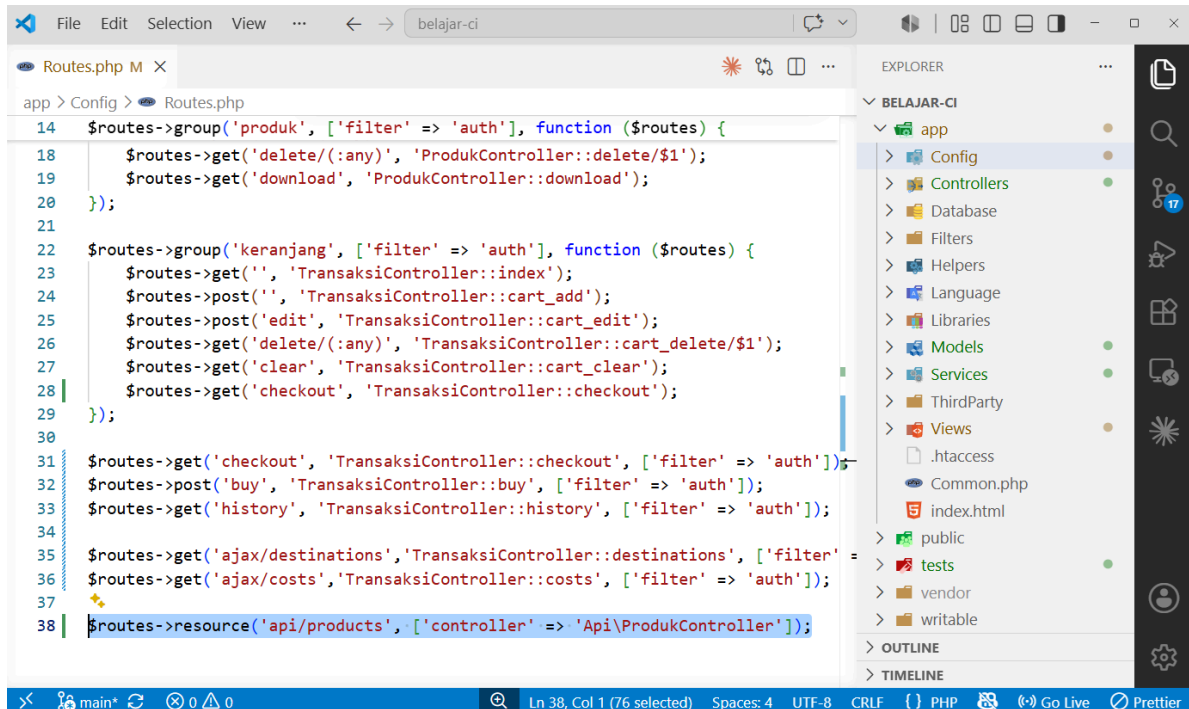


Pisahkan controller untuk webservice dengan meletakkannya pada subfolder (Api) pada folder app/Controllers. Pada file Api/ProdukController di dalamnya otomatis terdapat function **index()**, **show()**, **new()**, **create()**, **edit()**, **update()**, dan **delete()**, beserta penjelasannya. Fungsi-fungsi ini merupakan fungsi standard yang biasa digunakan dalam desain arsitektur **API** jenis **REST (Representational State Transfer)**.

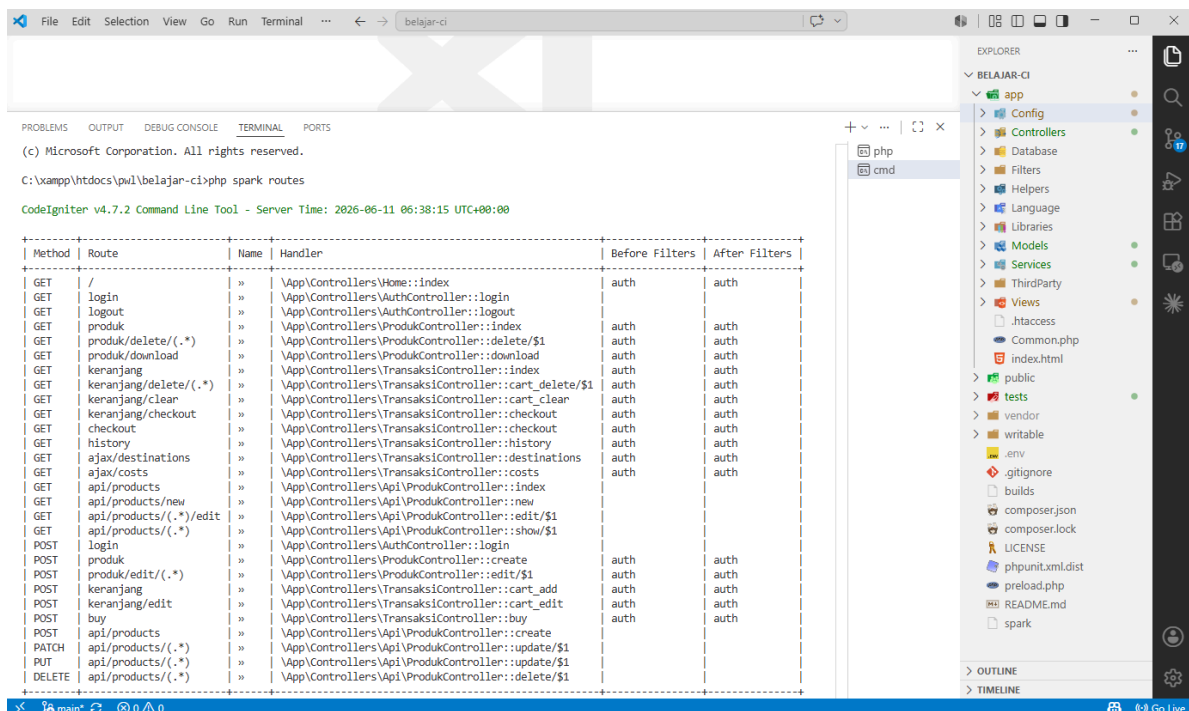


Sehingga memungkinkan aplikasi dari luar Toko melakukan aksi CRUD data produk yang dimiliki oleh project Toko, dengan menambahkan route ini pada file app/Config/Routes.php.


```
$routes->resource('api/products', ['controller' => 'Api\ProdukController']);
```



Coba cek lagi daftar route project Toko, akan bertambah dengan beberapa rute yang dibuat oleh route resource untuk data produk.



Dengan menambahkan sebaris routes dengan tipe resource pada file app/Config/Routes.php, otomatis terbentuk banyak route default yang biasa digunakan pada webservice, dengan method **GET, POST, PATCH, PUT, dan DELETE**.

Namun, semua routes restful tersebut tidak semuanya harus bisa digunakan/dilakukan. Webservice server bisa saja mengijinkan atau tidak mengijinkan dari masing-masing aksi tersebut. Misalnya aplikasi luar Toko hanya bisa menggunakan webservice untuk read data saja, tidak bisa insert/update/delete data. dll.

Sehingga endpoint pada project Toko yang tersedia pada webservice adalah

Fungsi	HTTP Method	Deskripsi
index	GET	Menampilkan daftar semua data endpoint : http://localhost:8080/api/products
show	GET	Menampilkan detail dari satu data berdasarkan ID tertentu. endpoint : http://localhost:8080/api/products/[id]
new	GET	Menampilkan form untuk membuat data baru (biasanya untuk web UI). endpoint : http://localhost:8080/api/products/new
create	POST	Menyimpan data baru ke database. endpoint : http://localhost:8080/api/products
edit	GET	Menampilkan form edit untuk data tertentu (biasanya untuk web UI). endpoint : http://localhost:8080/api/products/[id]/edit
update	PUT/PATCH	Memperbarui data yang sudah ada di database berdasarkan ID atau parameter tertentu. endpoint : http://localhost:8080/api/products/[id]
delete	DELETE	Menghapus data dari database berdasarkan ID atau parameter tertentu. endpoint : http://localhost:8080/api/products/[id]

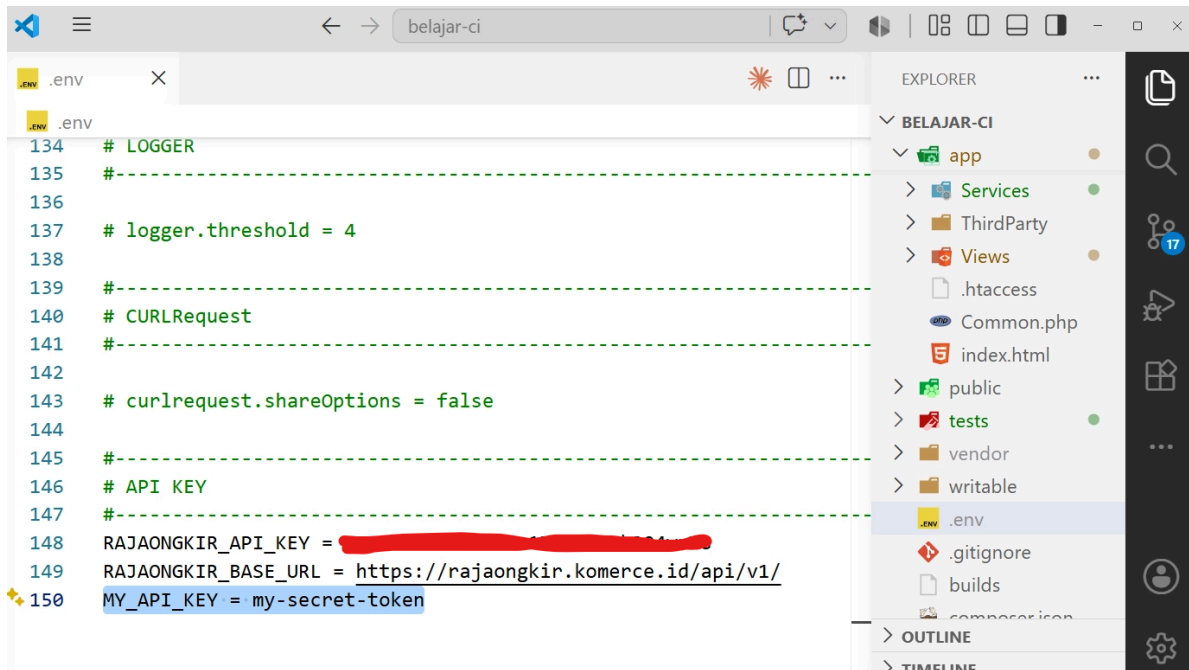
Fungsi `new` dan `edit` biasanya tidak dipakai di API, tapi lebih ke aplikasi web pada saat menampilkan tampilan form.

Untuk RESTful API lebih banyak menggunakan: `index` , `show` , `create` , `update` , dan `delete` .

- Ingat kembali pada tutorial sebelumnya saat menggunakan webservice dari Raja Ongkir. Untuk melakukan request data harus menyertakan sebuah API Key. Secara singkat, API key itu seperti "kunci digital" yang memastikan hanya pengguna terverifikasi yang bisa mengakses webservice dengan aman dan terkontrol.

Coba terapkan juga konsep tersebut pada kasus kali ini, jika ada aplikasi/sistem dari luar project ingin melakukan request ke endpoint Toko maka harus menyertakan sebuah API Key. Buka file `.env` lalu tambahkan string random (bebas) untuk API Key Toko.

```
MY_API_KEY = my-secret-token
```

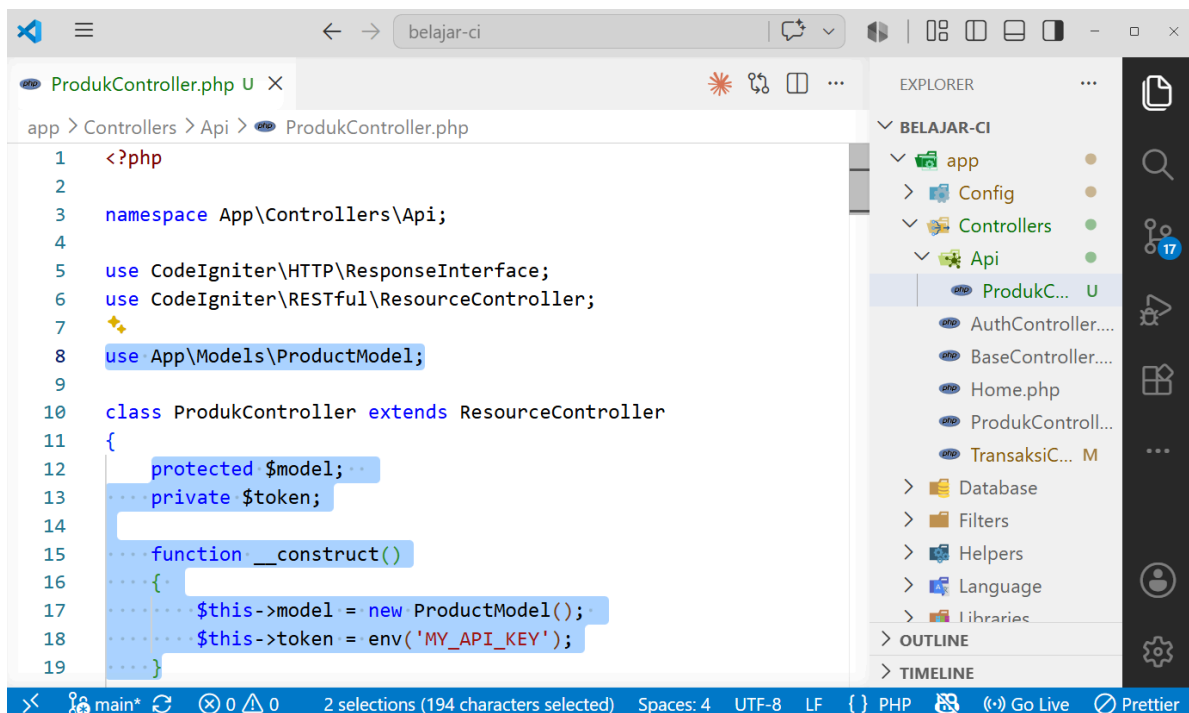


- Gunakan API KEY tersebut pada `app\Controllers\Api\ProdukController.php`.

```
use App\Models\ProductModel;

protected $model;
private $token;

function __construct()
{
    $this->model = new ProductModel();
    $this->token = env('MY_API_KEY');
}
```



Sehingga nanti request yang dibuat untuk endpoint project Toko harus menyertakan API KEY yang sama, misalnya seperti ini

```
//contoh request GET data produk
GET http://localhost:8080/api/products
```

```
Accept: application/json
Authorization: Bearer my-secret-token
```

- Buat fungsi untuk pengecekan API KEY yang dikirimkan melalui request dengan API KEY yang tersimpan pada file env.

Tambahkan pada file app\Controllers\Api\ProdukController.php.

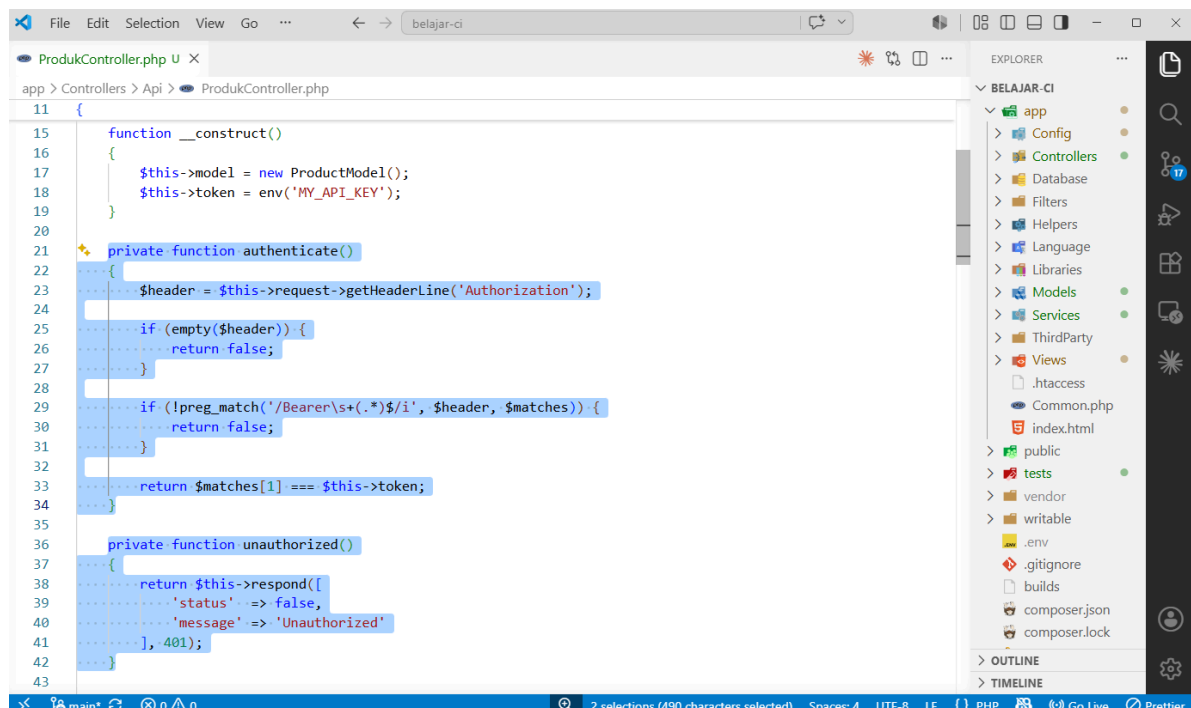
```
private function authenticate()
{
    $header = $this->request->getHeaderLine('Authorization');

    if (empty($header)) {
        return false;
    }

    if (!preg_match('/Bearer\s+(.*)$/i', $header, $matches)) {
        return false;
    }

    return $matches[1] === $this->token;
}

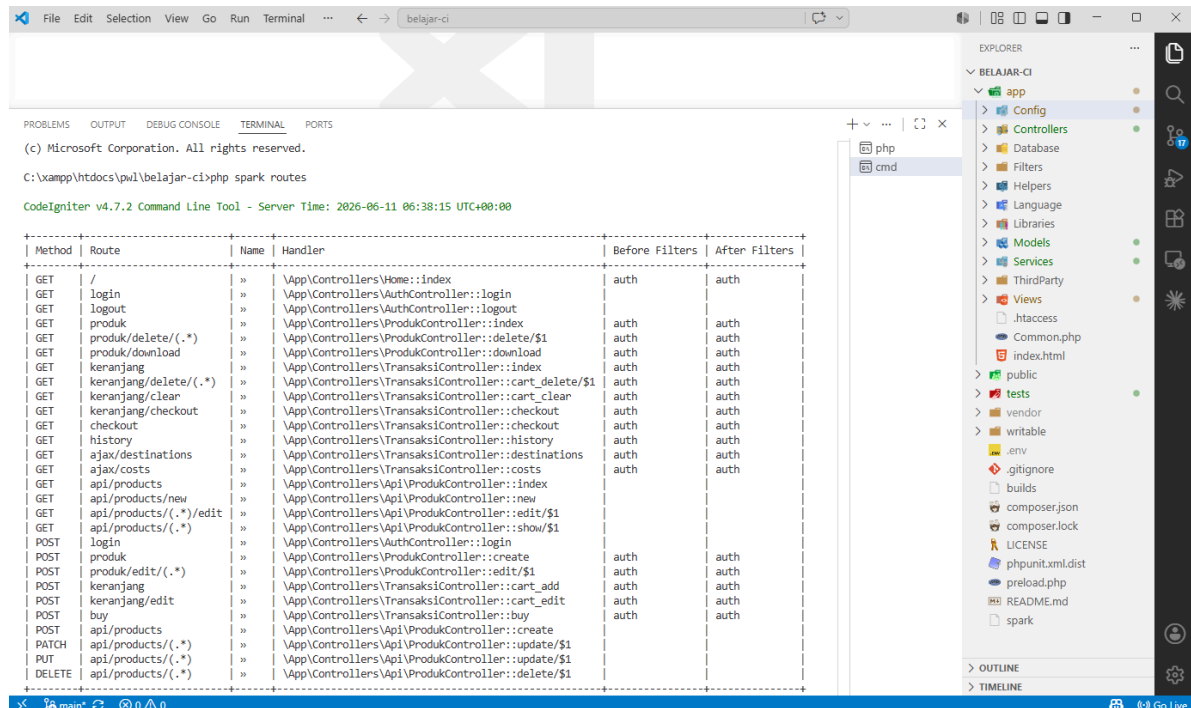
private function unauthorized()
{
    return $this->respond([
        'status' => false,
        'message' => 'Unauthorized'
    ], 401);
}
```



Fungsi ini akan digunakan untuk otentikasi pengecekan API KEY pada setiap fungsi di Api/ProdukController.

- Lanjutkan dengan melengkapi semua fungsi yang digunakan untuk endpoint data produk. Sesuai dengan rute yang sudah dibuat sebelumnya, untuk menampilkan data (GET) pada webservice menggunakan fungsi index() di Api/ProdukController.

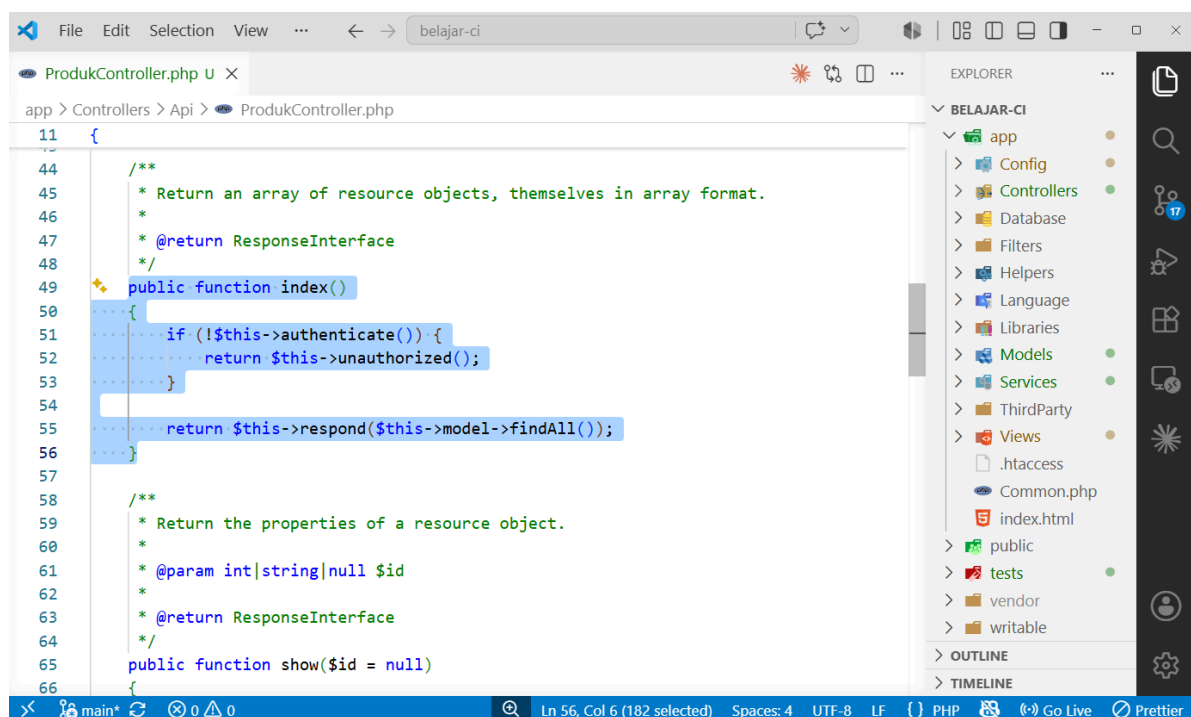
Endpoint : (GET) <http://localhost:8080/api/products>



Sesuaikan fungsi index() pada app\Controllers\Api\ProdukController.php. Tambahkan pengecekan otentikasi dengan fungsi authenticate() dan unauthorized(). Jika lolos maka berikan response yang berisi semua data produk dalam format JSON.

```
public function index()
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

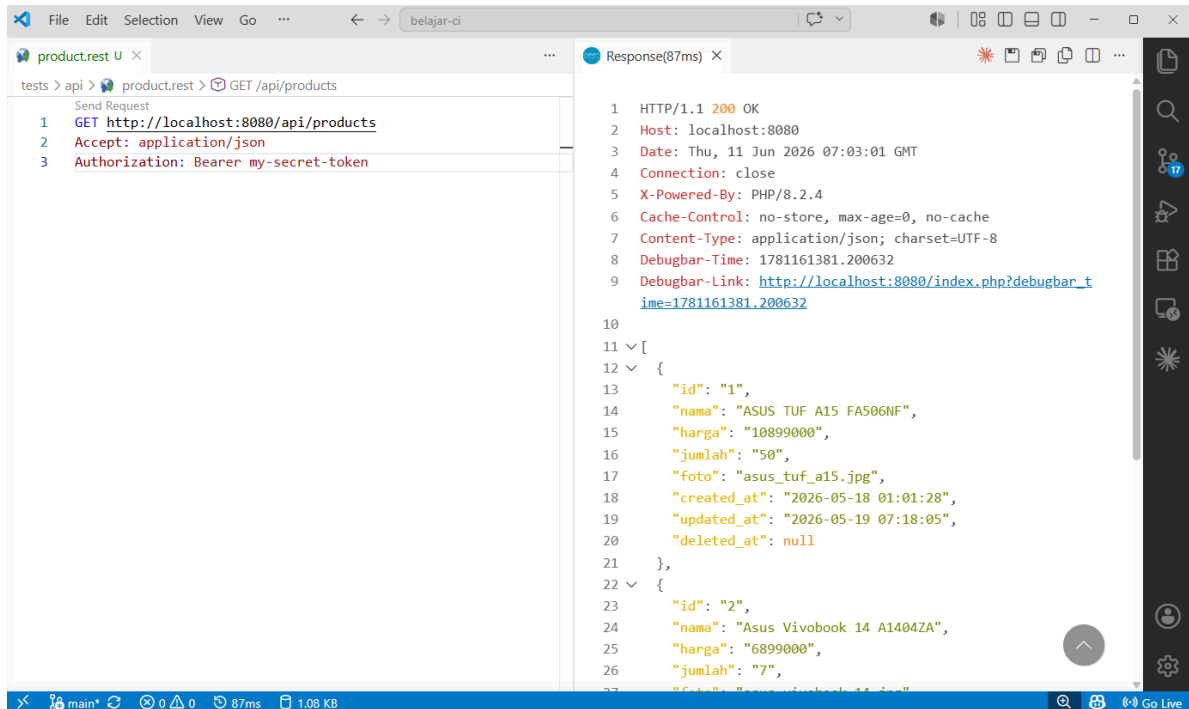
    return $this->respond($this->model->findAll());
}
```



Sama seperti pada saat akan menggunakan endpoint milik RajaOngkir, lakukan uji coba dulu endpoint project Toko ini dengan membuat sebuah file baru tests/api/product.rest. Isi dengan request GET dengan menyertakan API KEY pada header Authorization.

```
### index
GET http://localhost:8080/api/products
Accept: application/json
Authorization: Bearer my-secret-token
```

Untuk menjalankannya klik tulisan Send Request berwarna abu-abu pada file tests\api\product.rest (pastikan ada koneksi internet). Lalu response nya akan muncul di sebelah kanan.



- Endpoint untuk request GET data produk sudah bisa digunakan, namun data produk ditampilkan langsung semuanya pada response yang diberikan. Hal ini akan memberatkan jika pada database terdapat banyak data.

Oleh karena itu, endpoint untuk request GET data sering dibuat menggunakan pagination karena mengambil seluruh data sekaligus dapat menyebabkan masalah performa, penggunaan bandwidth, dan user experience yang kurang baik. Sesuaikan fungsi index () pada file app\Controllers\Api\ProdukController.php dengan menambahkan code untuk pagination.

```
public function index()
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

    $page = (int) ($this->request->getGet('page') ?? 1);
    $perPage = (int) ($this->request->getGet('per_page') ?? 10);

    $products = $this->model->paginate($perPage, 'default', $page);

    return $this->respond([
        'data' => $products,
        'pagination' => [
            'current_page' => $page,
            'per_page' => $perPage,
            'last_page' => $this->model->pager->getPageCount(),
            'total_data' => $this->model->pager->getTotal(),
            'has_next' => $page < $this->model->pager->getPageCount(),
            'has_prev' => $page > 1,
        ]
    ])
}
```

```
});
}
```

```

11 {
12
13     /**
14      * Return an array of resource objects, themselves in array format.
15      *
16      * @return ResponseInterface
17      */
18     public function index()
19     {
20         if (!$this->authenticate()) {
21             return $this->unauthorized();
22         }
23
24         $page = (int) ($this->request->getGet('page') ?? 1);
25         $perPage = (int) ($this->request->getGet('per_page') ?? 10);
26
27         $products = $this->model->paginate($perPage, 'default', $page);
28
29         return $this->respond([
30             'data' => $products,
31             'pagination' => [
32                 'current_page' => $page,
33                 'per_page' => $perPage,
34                 'last_page' => $this->model->pager->getPageCount(),
35                 'total_data' => $this->model->pager->getTotal(),
36                 'has_next' => $page < $this->model->pager->getPageCount(),
37                 'has_prev' => $page > 1,
38             ]
39         ]);
40     }
41 }

```

Sehingga request GET data bisa dilakukan dengan menyertakan parameter nomor halaman dan berapa jumlah data per halaman yang diinginkan.

Coba ubah request GET data pada file tests\api\product.rest, misalnya per halaman akan ditampilkan 1 data, maka request GET data untuk halaman 1 responsenya adalah seperti ini.

```

### index
GET http://localhost:8080/api/products?page=1&per_page=1
Accept: application/json
Authorization: Bearer my-secret-token

```

```

1 HTTP/1.1 200 OK
2 Host: localhost:8080
3 Date: Fri, 12 Jun 2026 03:56:17 GMT
4 Connection: close
5 X-Powered-By: PHP/8.2.4
6 Cache-Control: no-store, max-age=0, no-cache
7 Content-Type: application/json; charset=UTF-8
8 Debugbar-Time: 1781236577.589303
9 Debugbar-Link: http://localhost:8080/index.php?debugbar_time=1781236577.589303
10
11 {
12   "data": [
13     {
14       "id": "1",
15       "nama": "ASUS TUF A15 FA506NF",
16       "harga": "10899000",
17       "jumlah": "50",
18       "foto": "asus_tuf_a15.jpg",
19       "created_at": "2026-05-18 01:01:28",
20       "updated_at": "2026-05-19 07:18:05",
21       "deleted_at": null
22     }
23   ],
24   "pagination": {
25     "current_page": 1,
26     "per_page": 1,
27     "last_page": 3,
28     "total_data": 3,
29     "has_next": true,
30     "has_prev": false
31   }
32 }

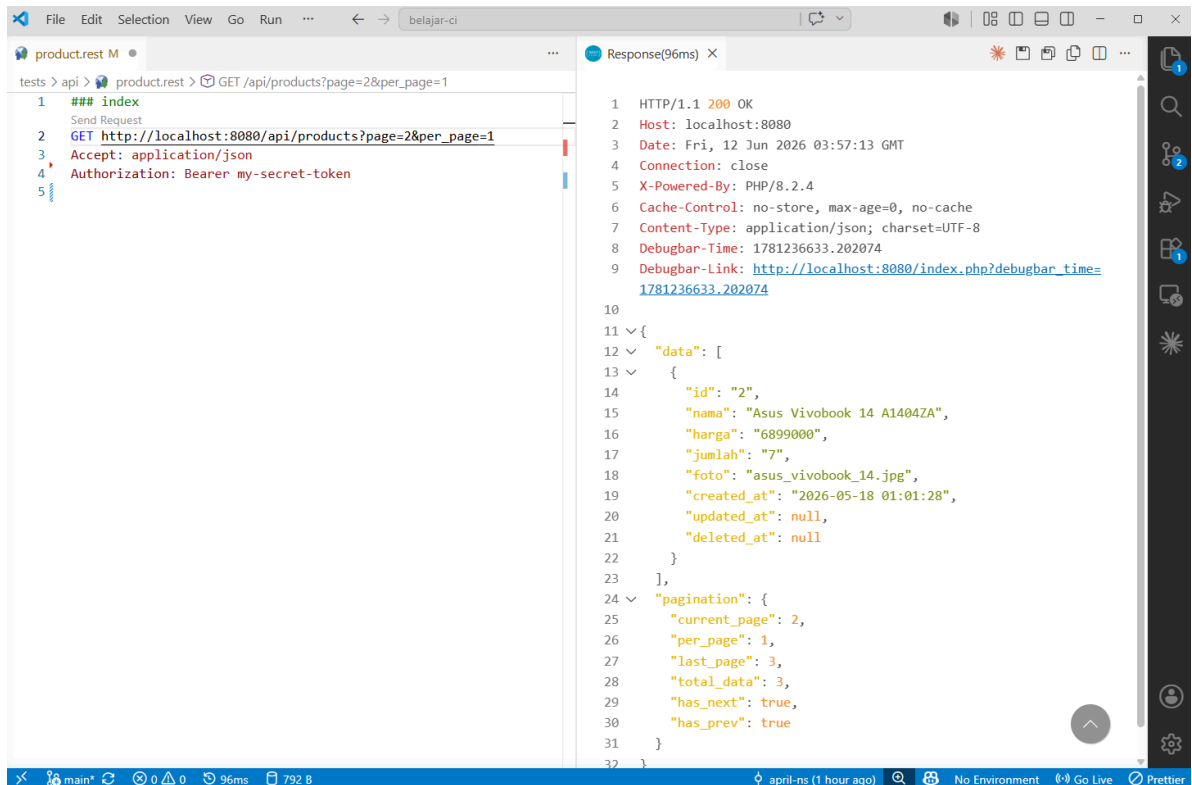
```

Pada detail data pagination, terdapat informasi saai total keseluruhan ada berapa data dan per page ditampilkan berapa data. Sehingga user tahu harus melakukan request halaman berapa saja setelah ini. Misalnya ada total 3 data, jika per page ditampilkan 1 data maka user harus melakukan request 3 kali yaitu page 1, page 2, dan page 3.

```
"pagination": {  
  "current_page": 1,  
  "per_page": 1,  
  "last_page": 3,  
  "total_data": 3,  
  "has_next": true,  
  "has_prev": false  
}
```

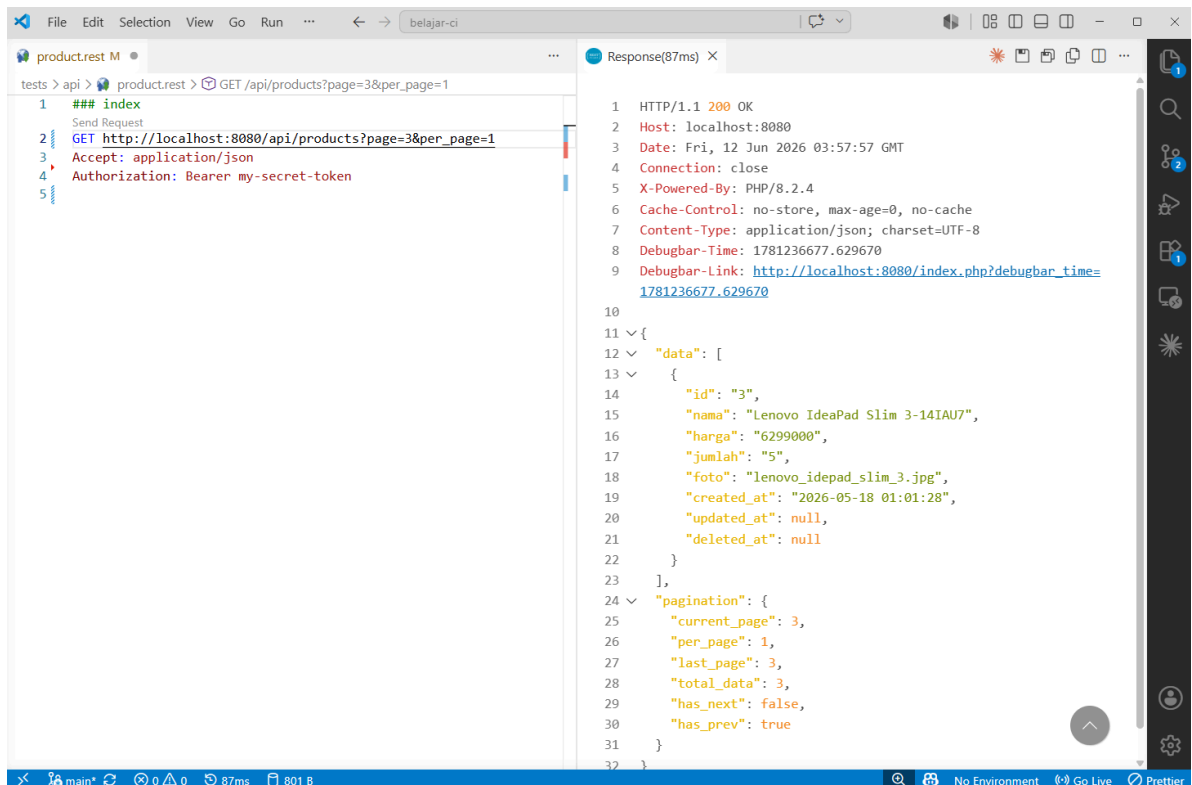
Lalu request GET data untuk halaman 2 responsenya adalah seperti ini.

```
### index  
GET http://localhost:8080/api/products?page=2&per_page=1  
Accept: application/json  
Authorization: Bearer my-secret-token
```

Terakhir, request GET data untuk halaman 3 responsenya adalah seperti ini.

```
### index
GET http://localhost:8080/api/products?page=3&per_page=1
Accept: application/json
Authorization: Bearer my-secret-token
```



Contoh nilai jumlah data per page dibuat kecil (1 data saja) karena data produk yang tersedia hanya sedikit, umumnya jumlah nilai data per page adalah 10-50 data.

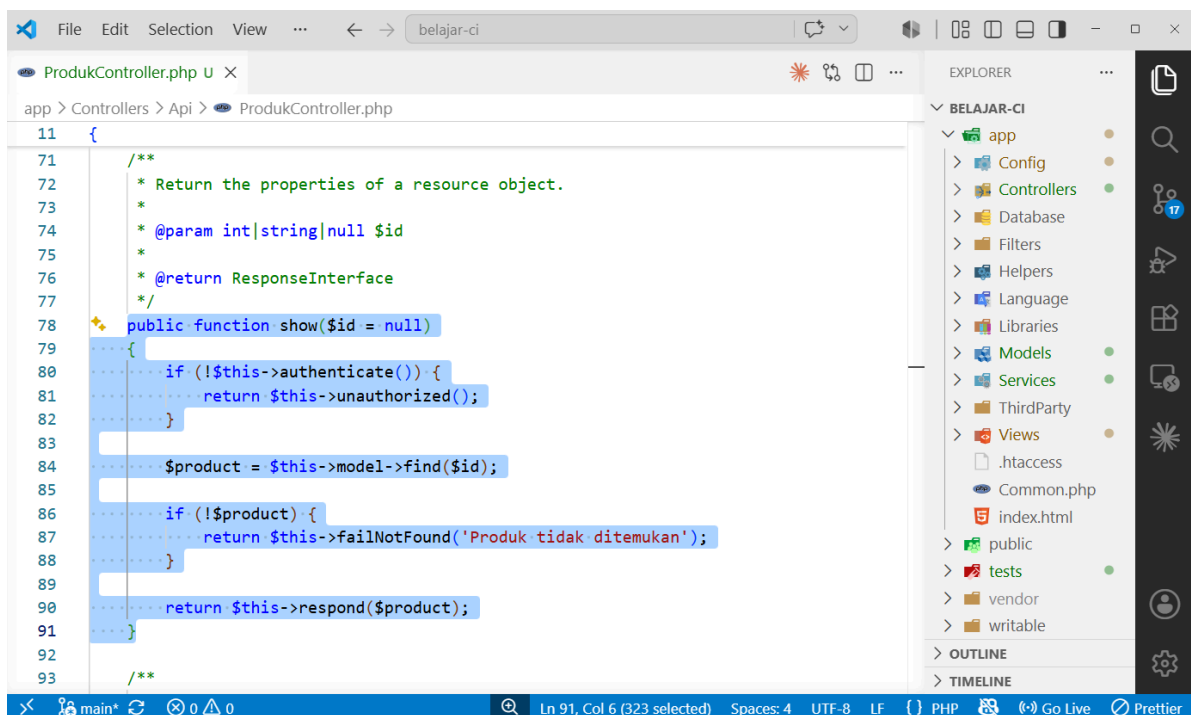
- Lanjutkan untuk endpoint yang lain, misalnya untuk menampilkan detail 1 data produk berdasarkan id nya. Endpoint : (GET) [http://localhost:8080/api/products/\[id\]](http://localhost:8080/api/products/[id]) Sesuaikan fungsi show() pada file app\Controllers\Api\ProdukController.php.

```
public function show($id = null)
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

    $product = $this->model->find($id);

    if (!$product) {
        return $this->failNotFound('Produk tidak ditemukan');
    }

    return $this->respond($product);
}
```



Untuk menambahkan data produk baru.

Endpoint : (POST) <http://localhost:8080/api/products>

Sesuaikan fungsi create() pada file app\Controllers\Api\ProdukController.php.

```
public function create()
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

    $data = $this->request->getJSON(true);

    $this->model->insert($data);

    return $this->respondCreated([
        'message' => 'Produk berhasil ditambahkan'
    ]);
}
```

Untuk mengubah data produk berdasarkan id nya.

Endpoint : (PUT/PATCH) [http://localhost:8080/api/products/\[id\]](http://localhost:8080/api/products/[id])

Sesuaikan fungsi update() pada file app\Controllers\Api\ProdukController.php.

```
public function update($id = null)
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

    if (!$this->model->find($id)) {
        return $this->failNotFound('Produk tidak ditemukan');
    }

    $data = $this->request->getJSON(true);

    $this->model->update($id, $data);

    return $this->respond([
        'message' => 'Produk berhasil diperbarui'
    ]);
}
```

Untuk menghapus data produk berdasarkan id nya.

Endpoint : (DELETE) [http://localhost:8080/api/products/\[id\]](http://localhost:8080/api/products/[id])

Sesuaikan fungsi delete() pada file app\Controllers\Api\ProdukController.php.

```
public function delete($id = null)
{
    if (!$this->authenticate()) {
        return $this->unauthorized();
    }

    if (!$this->model->find($id)) {
        return $this->failNotFound('Produk tidak ditemukan');
    }

    $this->model->delete($id);

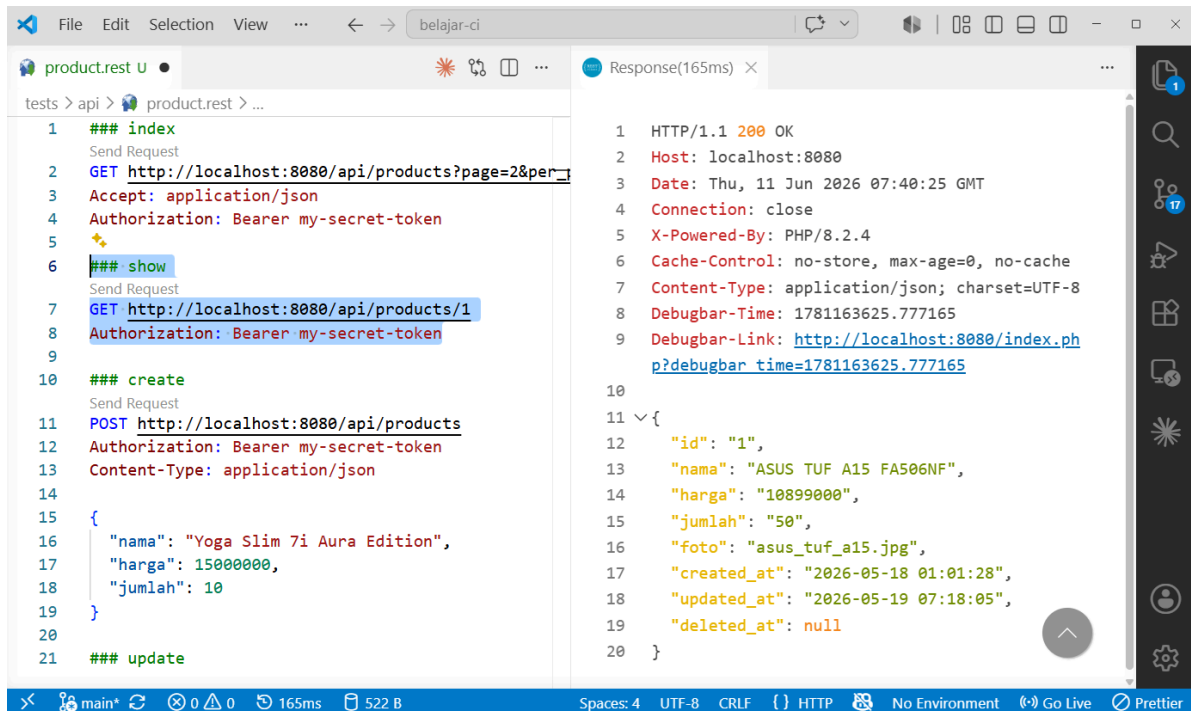
    return $this->respondDeleted([
        'message' => 'Produk berhasil dihapus'
    ]);
}
```

Untuk CRUD data produk melalui webservice ini dilakukan dengan mengabaikan data foto, sementara hanya fokus pada data nama, harga, dan jumlah produk saja.

- Tambahkan code untuk uji coba request GET data produk berdasarkan id nya pada file tests\api\product.rest.

```
### show
GET http://localhost:8080/api/products/1
Authorization: Bearer my-secret-token
```

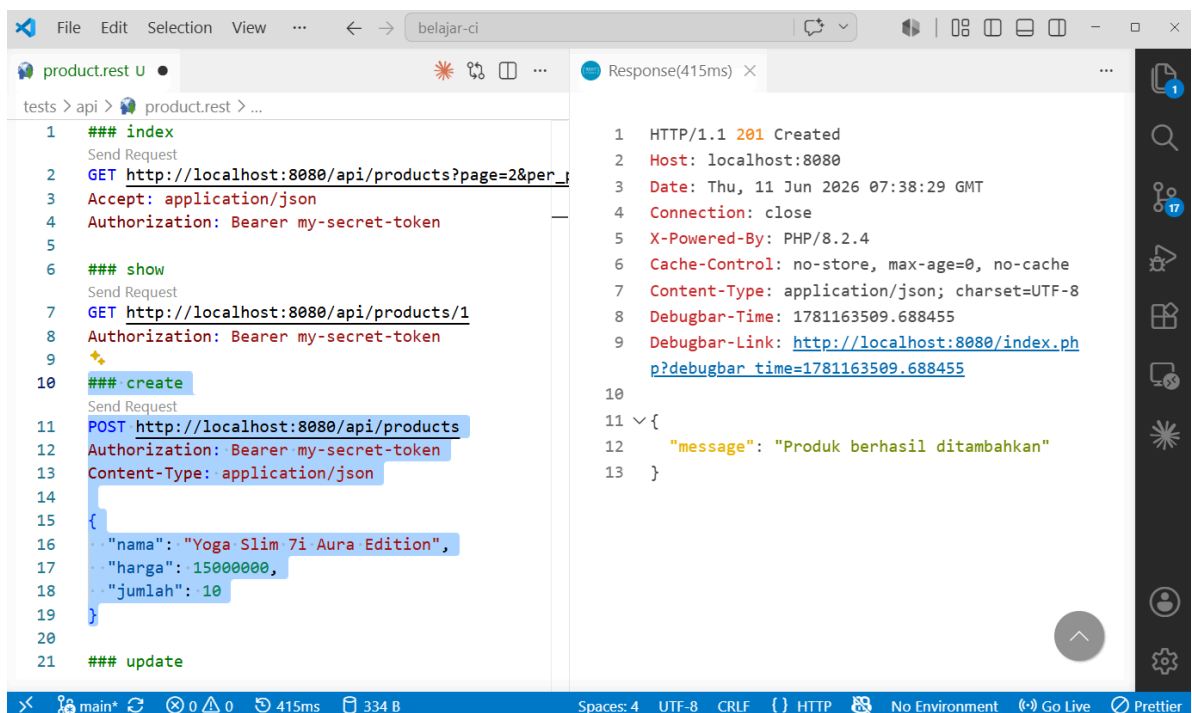
Menampilkan data produk yang memiliki id=1.



- Tambahkan code untuk uji coba request POST menambahkan data produk baru pada file tests\api\product.rest.

```
### create
POST http://localhost:8080/api/products
Authorization: Bearer my-secret-token
Content-Type: application/json

{
  "nama": "Yoga Slim 7i Aura Edition",
  "harga": 15000000,
  "jumlah": 10
}
```



Server: 127.0.0.1 » Database: db_ci4 » Table: product

Showing rows 0 - 4 (5 total, Query took 0.0045 seconds.)

SELECT * FROM `product`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	nama	harga	jumlah	foto	created_at	updated_at	deleted_at
☐ Edit ☐ Copy ☐ Delete	1	ASUS TUF A15 FA506NF	10899000	50	asus_tuf_a15.jpg	2026-05-18 01:01:28	2026-05-19 07:18:05	NULL
☐ Edit ☐ Copy ☐ Delete	2	Asus Vivobook 14 A1404ZA	6899000	7	asus_vivobook_14.jpg	2026-05-18 01:01:28	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	3	Lenovo IdeaPad Slim 3-14IAU7	6299000	5	lenovo_ideapad_slim_3.jpg	2026-05-18 01:01:28	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	9	Lenovo Ideapad D330	3500000	12	1779174897_ae0946891f7d1b917de.jpg	2026-05-19 07:14:57	2026-05-19 07:20:12	2026-05-19 07:20:12
☐ Edit ☐ Copy ☐ Delete	13	Yoga Slim 7i Aura Edition	15000000	10		2026-06-11 07:38:29	2026-06-11 07:38:29	NULL

- Tambahkan code untuk uji coba request PUT data produk berdasarkan id nya pada file tests\api\product.rest. Request ini dilakukan jika client ingin merubah nilai semua field data produk tertentu.

```
### update all field
PUT http://localhost:8080/api/products/13
Authorization: Bearer my-secret-token
Content-Type: application/json

{
  "nama": "Yoga Slim 7i Aura Editions",
  "harga": 25000000,
  "jumlah": 20
}
```

Mengubah nilai field nama, harga, jumlah pada produk yang memiliki id=13.

product.rest U •

tests > api > product.rest > ...

```
10 ### create
11 Send Request
12 POST http://localhost:8080/api/products
13 Authorization: Bearer my-secret-token
14 Content-Type: application/json
15 {
16   "nama": "Yoga Slim 7i Aura Edition",
17   "harga": 15000000,
18   "jumlah": 10
19 }
20
21 ### update all field
22 Send Request
23 PUT http://localhost:8080/api/products/13
24 Authorization: Bearer my-secret-token
25 Content-Type: application/json
26 {
27   "nama": "Yoga Slim 7i Aura Editions",
28   "harga": 25000000,
29   "jumlah": 20
30 }
31
```

Response(108ms) X

```
1 HTTP/1.1 200 OK
2 Host: localhost:8080
3 Date: Thu, 11 Jun 2026 07:44:47 GMT
4 Connection: close
5 X-Powered-By: PHP/8.2.4
6 Cache-Control: no-store, max-age=0, no-cache
7 Content-Type: application/json; charset=UTF-8
8 Debugbar-Time: 1781163887.405841
9 Debugbar-Link: http://localhost:8080/index.php?debugbar_time=1781163887.405841
10
11 {
12   "message": "Produk berhasil diperbarui"
13 }
```

Spaces: 4 UTF-8 CRLF {} HTTP No Environment Go Live Prettier

Server: 127.0.0.1 » Database: db_ci4 » Table: product

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.)

SELECT * FROM `product`

Filter rows: Search this table Sort by key: None

	id	nama	harga	jumlah	foto	created_at	updated_at	deleted_at
☐	1	ASUS TUF A15 FA506NF	10899000	50	asus_tuf_a15.jpg	2026-05-18 01:01:28	2026-05-19 07:18:05	NULL
☐	2	Asus Vivobook 14 A1404ZA	6899000	7	asus_vivobook_14.jpg	2026-05-18 01:01:28	NULL	NULL
☐	3	Lenovo IdeaPad Slim 3-14IAU7	6299000	5	lenovo_ideapad_slim_3.jpg	2026-05-18 01:01:28	NULL	NULL
☐	9	Lenovo Ideapad D330	3500000	12	1779174897_ae0946891ff7d1b917de.jpg	2026-05-19 07:14:57	2026-05-19 07:20:12	2026-05-19 07:20:12
☐	13	Yoga Slim 7i Aura Editions	25000000	20		2026-06-11 07:38:29	2026-06-11 07:44:47	NULL

- Tambahkan code untuk uji coba request PATCH data produk berdasarkan id nya pada file tests\api\product.rest. Request ini dilakukan jika client ingin merubah nilai beberapa field data produk tertentu.

```
### update some field
PATCH http://localhost:8080/api/products/13
Authorization: Bearer my-secret-token
Content-Type: application/json

{
  "jumlah": 25
}
```

Mengubah nilai field jumlah pada produk yang memiliki id=13.

product.rest U

tests > api > product.rest > ...

```
11 POST http://localhost:8080/api/products
19 }
20
21 ### update all field
22 Send Request
23 PUT http://localhost:8080/api/products/13
24 Authorization: Bearer my-secret-token
25 Content-Type: application/json
26 {
27   "nama": "Yoga Slim 7i Aura Editions",
28   "harga": 25000000,
29   "jumlah": 20
30 }
31
32 ### update some field
33 Send Request
34 PATCH http://localhost:8080/api/products/13
35 Authorization: Bearer my-secret-token
36 Content-Type: application/json
37 {
38   "jumlah": 25
39 }
```

Response(86ms)

```
1 HTTP/1.1 200 OK
2 Host: localhost:8080
3 Date: Thu, 11 Jun 2026 07:46:22 GMT
4 Connection: close
5 X-Powered-By: PHP/8.2.4
6 Cache-Control: no-store, max-age=0, no-cache
7 Content-Type: application/json; charset=UTF-8
8 Debugbar-Time: 1781163982.003424
9 Debugbar-Link: http://localhost:8080/index.php?debugbar_time=1781163982.003424
10
11 {
12   "message": "Produk berhasil diperbarui"
13 }
```

main* 0 0 86ms 333 B Spaces: 4 UTF-8 CRLF HTTP No Environment Go Live Prettier

Server: 127.0.0.1 > Database: db_ci4 > Table: product

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds)

SELECT * FROM `product`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	nama	harga	jumlah	foto	created_at	updated_at	deleted_at
☐	1	ASUS TUF A15 FA506NF	10899000	50	asus_tuf_a15.jpg	2026-05-18 01:01:28	2026-05-19 07:18:05	NULL
☐	2	Asus Vivobook 14 A1404ZA	6899000	7	asus_vivobook_14.jpg	2026-05-18 01:01:28	NULL	NULL
☐	3	Lenovo IdeaPad Slim 3-14IAU7	6299000	5	lenovo_ideapad_slim_3.jpg	2026-05-18 01:01:28	NULL	NULL
☐	9	Lenovo Ideapad D330	3500000	12	1779174897_ae0946891ff7d1b917de.jpg	2026-05-19 07:14:57	2026-05-19 07:20:12	2026-05-19 07:20:12
☐	13	Yoga Slim 7i Aura Editions	25000000	25		2026-06-11 07:38:29	2026-06-11 07:46:21	NULL

- Tambahkan code untuk uji coba request DELETE data produk berdasarkan id nya pada file tests\api\product.rest.

```
### delete
DELETE http://localhost:8080/api/products/13
Authorization: Bearer my-secret-token
```

Menghapus data produk yang memiliki id=13.

product.rest u

tests > api > product.rest > ...

```
22 PUT http://localhost:8080/api/products/13
23 Content-Type: application/json
24
25 {
26   "nama": "Yoga Slim 7i Aura Editions",
27   "harga": 25000000,
28   "jumlah": 20
29 }
30
31
32 ### update some field
33 Send Request
34 PATCH http://localhost:8080/api/products/13
35 Authorization: Bearer my-secret-token
36 Content-Type: application/json
37 {
38   "jumlah": 25
39 }
40
41 ### delete
42 Send Request
43 DELETE http://localhost:8080/api/products/13
44 Authorization: Bearer my-secret-token
```

Response(92ms)

```
1 HTTP/1.1 200 OK
2 Host: localhost:8080
3 Date: Thu, 11 Jun 2026 07:47:38 GMT
4 Connection: close
5 X-Powered-By: PHP/8.2.4
6 Cache-Control: no-store, max-age=0, no-cache
7 Content-Type: application/json; charset=UTF-8
8 Debugbar-Time: 1781164058.085242
9 Debugbar-Link: http://localhost:8080/index.php?debugbar_time=1781164058.085242
10
11 {
12   "message": "Produk berhasil dihapus"
13 }
```

main* 0 92ms 330 B Spaces: 4 UTF-8 CRLF {} HTTP No Environment Go Live Prettier

Server: 127.0.0.1 > Database: db_ci4 > Table: product

Showing rows 0 - 4 (5 total, Query took 0.0003 seconds)

SELECT * FROM `product`

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	id	nama	harga	jumlah	foto	created_at	updated_at	deleted_at
☐	1	ASUS TUF A15 FA506NF	10899000	50	asus_tuf_a15.jpg	2026-05-18 01:01:28	2026-05-19 07:18:05	NULL
☐	2	Asus Vivobook 14 A1404ZA	6899000	7	asus_vivobook_14.jpg	2026-05-18 01:01:28	NULL	NULL
☐	3	Lenovo IdeaPad Slim 3-14IAU7	6299000	5	lenovo_ideapad_slim_3.jpg	2026-05-18 01:01:28	NULL	NULL
☐	9	Lenovo Ideapad D330	3500000	12	1779174897_ae0946891ff7d1b917de.jpg	2026-05-19 07:14:57	2026-05-19 07:20:12	2026-05-19 07:20:12
☐	13	Yoga Slim 7i Aura Editions	25000000	25		2026-06-11 07:38:29	2026-06-11 07:47:38	2026-06-11 07:47:38

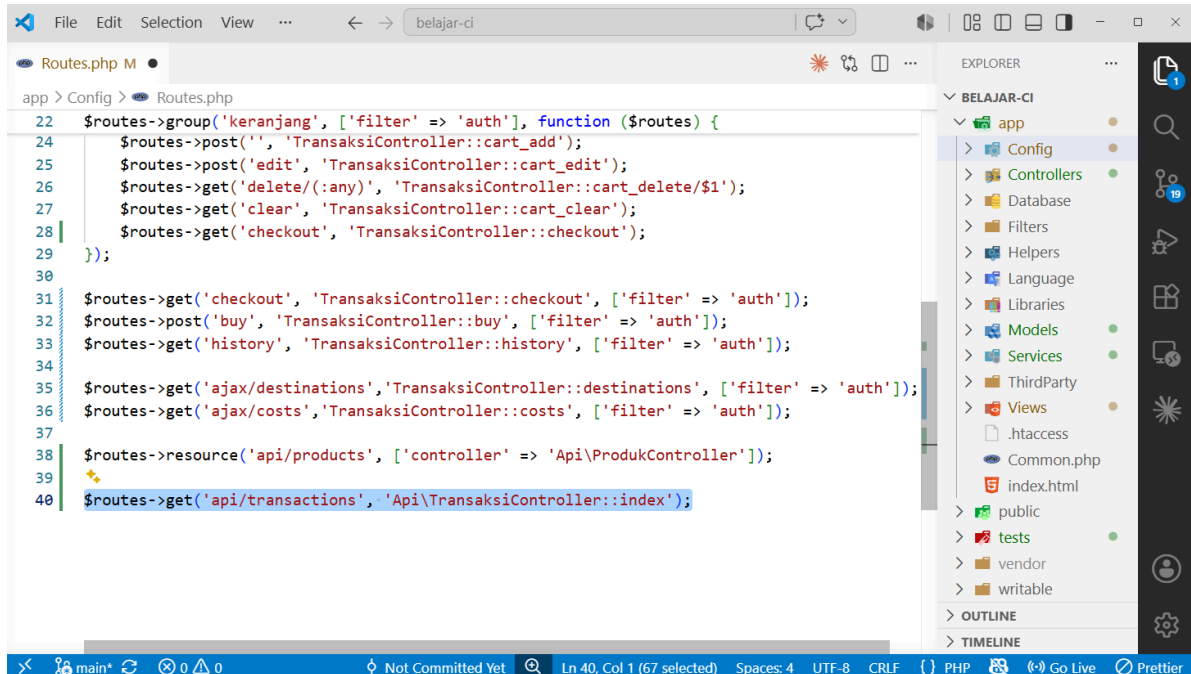
Sampai sini, wesbservice untuk data produk sudah selesai dibuat dan diuji coba.

Membuat endpoint data transaksi

- Webservice kedua yang akan dibuat adalah endpoint untuk mendapatkan data transaksi pada range tanggal tertentu. Karena endpoint ini hanya digunakan untuk request GET data (tidak digunakan untuk CRUD data) maka buat saja route seperti biasa (bukan yang jenis resource).

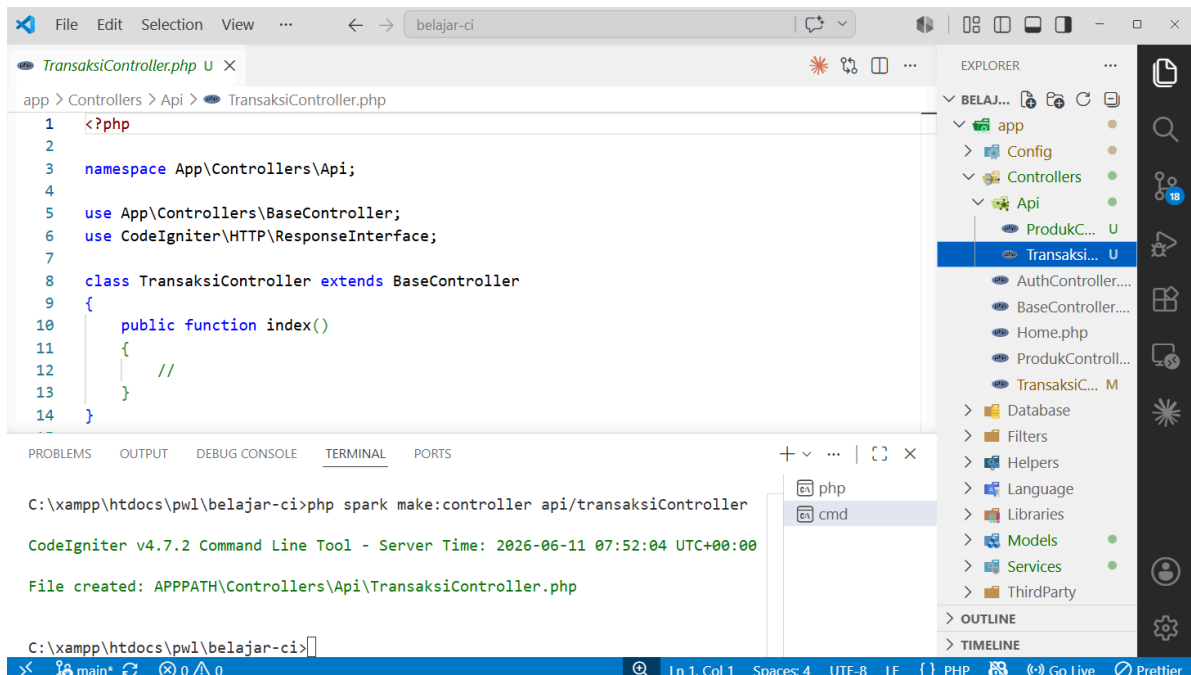
Tambahkan dulu rutanya pada file app\Config\Routes.php.

```
$routes->get('api/transactions', 'Api\TransaksiController::index');
```



- Kemudian buat controllernya sesuai dengan route yang barusan dibuat, arahkan lokasi controller ke dalam folder app\Controllers\Api.

```
php spark make:controller api/transaksiController
```



- Isi file app\Controllers\Api\TransaksiController.php kurang lebih sama dengan endpoint produk sebelumnya. Bedanya pada file ini menggunakan 2 file model yaitu TransactionModel dan TransactionDetailModel. Kemudian di file ini hanya menyediakan 1 fungsi index() untuk logic pembuatan response untuk request GET data transaksi untuk range tanggal tertentu.


```

<?php

namespace App\Controllers\Api;

use App\Controllers\BaseController;
use CodeIgniter\HTTP\ResponseInterface;

use App\Models\TransactionModel;
use App\Models\TransactionDetailModel;

class TransaksiController extends BaseController
{
    protected $transactionModel;
    protected $transactionDetailModel;
    private $token;

    function __construct()
    {
        $this->transactionModel = new TransactionModel();
        $this->transactionDetailModel = new TransactionDetailModel();
        $this->token = env('MY_API_KEY');
    }

    private function authenticate()
    {
        $header = $this->request->getHeaderLine('Authorization');

        if (empty($header)) {
            return false;
        }

        if (!preg_match('/Bearer\s+(.*)$/i', $header, $matches)) {
            return false;
        }

        return $matches[1] === $this->token;
    }

    private function unauthorized()
    {
        return $this->response
            ->setStatusCode(401)
            ->setJSON([
                'status' => false,
                'message' => 'Unauthorized'
            ]);
    }

    public function index()
    {
        if (! $this->authenticate()) {
            return $this->unauthorized();
        }

        $start = $this->request->getGet('start');
        $end = $this->request->getGet('end');

        $page = (int) ($this->request->getGet('page') ?? 1);
        $perPage = (int) ($this->request->getGet('per_page') ?? 10);

        // Query transaksi
        $query = $this->transactionModel->orderBy('created_at', 'DESC');

        if ($start && $end) {

```

```

        $query->where('created_at >=', $start)->where('created_at <=', $end);
    }

    // Pagination
    $transactions = $query->paginate($perPage, 'default', $page);

    // Ambil ID transaksi
    $transactionIds = [];

    if (!empty($transactions)) {
        $transactionIds = array_column($transactions, 'id');
    }

    // Ambil detail + produk
    $products = [];

    if (!empty($transactionIds)) {
        $products = $this->transactionDetailModel->getProductsByTransactionIds($transaction
    }

    // Inject detail ke transaksi
    foreach ($transactions as $key => $trx) {
        $transactions[$key]['details'] = $products[$trx['id']] ?? [];
    }

    // Pagination info
    $pager = $this->transactionModel->pager;

    return $this->response->setJSON([
        'filter' => [
            'start' => $start,
            'end'   => $end,
        ],
        'data' => $transactions,
        'pagination' => [
            'current_page' => $page,
            'per_page'     => $perPage,
            'last_page'    => $pager->getPageCount(),
            'total_data'   => $pager->getTotal(),
            'has_next'     => $page < $pager->getPageCount(),
            'has_prev'     => $page > 1,
        ]
    ]);
}
}
}

```

- Lakukan uji coba endpoint data transaksi dengan membuat sebuah file baru tests/api/transaction.rest. Isi dengan request GET untuk dengan menyertakan API KEY pada header Authorization.

```

GET http://localhost:8080/api/transactions?start=2026-06-01&end=2026-06-30&page=1&per_page=10
Accept: application/json
Authorization: Bearer my-secret-token

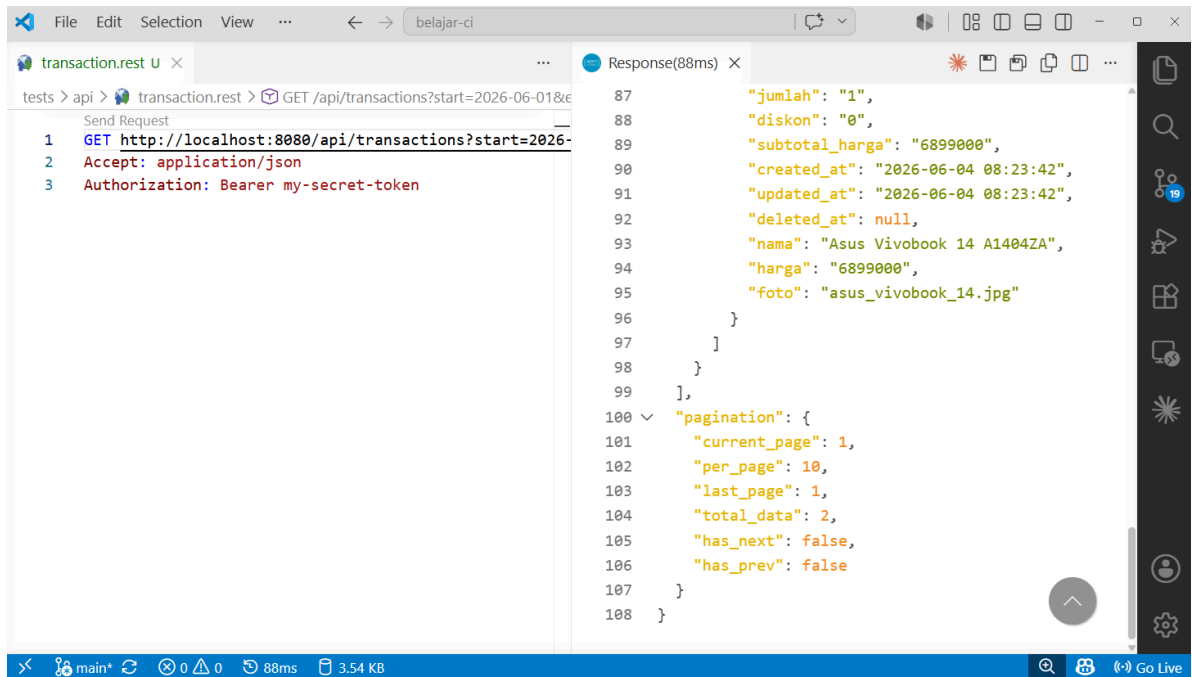
```

```
File Edit Selection View ... < -> belajar-ci
transaction.rest U x ... Response(88ms) x
tests > api > transaction.rest > GET /api/transactions?start=2026-06-01&e
Send Request
1 GET http://localhost:8080/api/transactions?start=2026-
2 Accept: application/json
3 Authorization: Bearer my-secret-token

1 HTTP/1.1 200 OK
2 Host: localhost:8080
3 Date: Thu, 11 Jun 2026 08:28:50 GMT
4 Connection: close
5 X-Powered-By: PHP/8.2.4
6 Cache-Control: no-store, max-age=0, no-cache
7 Content-Type: application/json; charset=UTF-8
8 Debugbar-Time: 1781166530.973584
9 Debugbar-Link: http://localhost:8080/index.php?
debugbar time=1781166530.973584
10
11 {
12   "filter": {
13     "start": "2026-06-01",
14     "end": "2026-06-30"
15   },
16   "data": [
17     {
18       "id": "2",
19       "username": "spuspita",
20       "total_harga": "35796000",
21       "alamat": "Jl. Imam Bonjol 207",
```

```
File Edit Selection View Go Run ... < -> belajar-ci
transaction.rest U x ... Response(88ms) x
tests > api > transaction.rest > GET /api/transactions?start=2026-06-01&end=2026-06-30&e
Send Request
1 GET http://localhost:8080/api/transactions?start=2026-06-01&end=2026-06-30&e
2 Accept: application/json
3 Authorization: Bearer my-secret-token

16 "data": [
17   {
18     "id": "2",
19     "username": "spuspita",
20     "total_harga": "35796000",
21     "alamat": "Jl. Imam Bonjol 207",
22     "ongkir": "200000",
23     "status": "0",
24     "created_at": "2026-06-04 08:32:52",
25     "updated_at": "2026-06-04 08:32:52",
26     "deleted_at": null,
27     "details": [
28       {
29         "id": "3",
30         "transaction_id": "2",
31         "product_id": "1",
32         "jumlah": "2",
33         "diskon": "0",
34         "subtotal_harga": "21798000",
35         "created_at": "2026-06-04 08:32:52",
36         "updated_at": "2026-06-04 08:32:52",
37         "deleted_at": null,
38         "nama": "ASUS TUF A15 FA506NF",
39         "harga": "10899000",
40         "foto": "asus_tuf_a15.jpg"
41       },
42       {
43         "id": "4",
44         "transaction_id": "2",
```



- Jika sudah berhasil dijalankan di localhost, jangan lupa push perubahan dan tambahan yang dilakukan ke repository github masing-masing.

```
git add .
git commit -m "menambahkan api data produk dan transaksi"
git push origin main
```